P0

100 Points Possible

Add Comment

Details

Objective

Programming refreshment, practice with coding and architectural standards, practice with trees, traversals, command line arguments, and file IO.

If using generative AI, you must "own" the code, and anyone can be asked for a demonstration with explanations to get any credit. You may generate code for some aspects, but unlikely all at once, so you would have to put the solutions together to create a final solution.

Future projects will use the same or similar requirements invocation. If you code this properly and with good modularity, you will be able to easily reuse.

Submission command

```
/accounts/classes/janikowc/submitProject/submit_cs4280s1_P0 SubmitFileOrDirectory
```

Submission on Delmar will be illustrated before the submission deadline. If you develop your program in another environment, make sure to transfer the files properly (text transfer for sources/headers) and to retest. The project is marked as "paper submission" to simplify this in Canvas.

Specification

Write a program to build a tree for a file input, and then print the tree using three different traversals. The program will be invoked as

Invocation

P0 [*file*]

where ***file*** is an optional argument. If the ***file*** argument is not given, the program reads data from the keyboard.
If the argument is given, the program reads the data file ***file*.fs25s1**. (note that ***file*** is any valid name and the extension is implicit - it is NOT allowed on the command line).
Programs not properly implementing invocation can get max 50% as they will not be tested, at most they would be inspected.

Invocation Illustrations

Without optional argument

P0 // read from the keyboard until simulated EOF
P0 < somefile // same as above except redirecting from **somefile** instead of keyboard, this tests keyboard input. **somefile** must be the entire filename including the otherwise implicit extension, and this is the same as above except the shell submits the file to the program through the standard input

With optional argument

P0 somename // read from somename.fs25s1

Requirements

Assume you do not know the size of the input file.
Assume the input data is all strings (printable ASCII) separated with standard WS (white space) separators (space, tab, new line).
If the input file is not readable for whatever reason, or command line arguments are not correct, the program must abort with an appropriate message.

The program will read the data left to right and put them into a tree, which is a binary search tree (BST). The data is all strings made up of letters and digits only. You may assume clean data, +10% for validation - a warning message on any bad datum and then skipping that datum.

The data is placed in the binary search tree using the definition that any datum repeated in the file a number of times is larger than another datum that was repeated a smaller number of times. Data repeated the same number of times is interpreted as equivalent data from the ordering perspective (thus falling in the same node).

Tree is never balanced nor restructured other than growing new nodes as needed, and the tree is constructed using the first occurrence of each string.

A node retains each repeated datum just once.

The program will subsequently output 3 files corresponding to 3 traversals, named ***somename*.preorder**, ***somename*.inorder** and ***somename*.postorder**. Note that ***somename*** is the base name of the input file if given, and it is the constant string **out** if that argument is not not given.

Treversals
- preorder
- inorder
- postoder

Printing in traversal

a node will print starting with indentation of 2x depth (root is at depth 0) then the node's number (the frequency for the node's data) followed by the character ':' and WS followed by the list of strings from the node separated by WS (retaining the original ordering as according to appearance in the input file).

Structural Requirements

The main function should be in the file **P0.c** (in C, P0.cpp in C++), with any helpers helping it processing the argument or generate errors in the same file. Then, you should have functions

buildTree()
printInorder()
printPreorder()
printPostorder()

(signatures up to you) implemented in the same file **tree.c**/**tree.cpp** with **tree.h** as needed. If you have a type definition for the node, it should be in a separate header file **node.h**.

Traversals taken from the 3130 textbook:

Preorder:

process root
process children left to right

Inorder:

process left child
process root
proccess right child

Postorder:

process children left to right
process root

Grading

15% standards: 5% coding and 10% architectural
15% keyboard input
10% proper errors of wrong arguments or files
60% proper outputs
+10% for data validation, see above

Projects not compiling, or not running under the required invocation, can at most get partial grade by inspection, with the max of 50%.